Week 4 RNN

This practice demonstrates the future price prediction for diffrent stocks using recurrent neural networks in tensorflow.

In a traditional neural network, we assume that all inputs (and outputs) are independent of each other. But for many tasks that are a very bad idea. If you want to predict the next word in a sentence you better know which words came before it. RNNs are called recurrent because they perform the same task for every element of a sequence, with the output being dependent on the previous computations. RNN has a “memory” which remembers all information about what has been calculated. It uses the same parameters for each input as it performs the same task on all the inputs or hidden layers to produce the output. This reduces the complexity of parameters, unlike other neural networks.

Architecture of RNN

The key feature of RNNs is their ability to have context about the information they are dealing with, as they can utilize information from previous time steps or positions in the sequence. The basic architecture of an RNN consists of recurrent units or cells that form a chain-like structure. Each cell takes an input, produces an output, and has a hidden state that serves as its memory. The hidden state from one time-step is fed back into the cell as input for the next time step, allowing the network to capture and utilize information from previous steps.

No description has been provided for this image

Long Short-Term Memory Networks (LSTMs):

LSTMs are a special type of RNN designed to address the limitations of standard RNNs. They use a more complex architecture with gates (input, forget, and output gates) to control the flow of information. These gates enable LSTMs to:

1. Retain long-term dependencies effectively.
2. Avoid the vanishing gradient problem.

LSTMs are widely used in tasks requiring memory over extended sequences, such as stock price prediction, speech recognition, and text generation. While more computationally intensive than RNNs, LSTMs are generally more powerful and robust for complex sequential tasks.

1. Packages

In [15]:
```
import numpy as np
import pandas as pd
import math
import sklearn
```

```
import sklearn.preprocessing
import datetime
import os
import matplotlib.pyplot as plt
import tensorflow as tf
print(tf.__version__)
```

```
2.19.0
```

2. Analyze data

- load stock prices from csv file
- analyze data

In [17]:

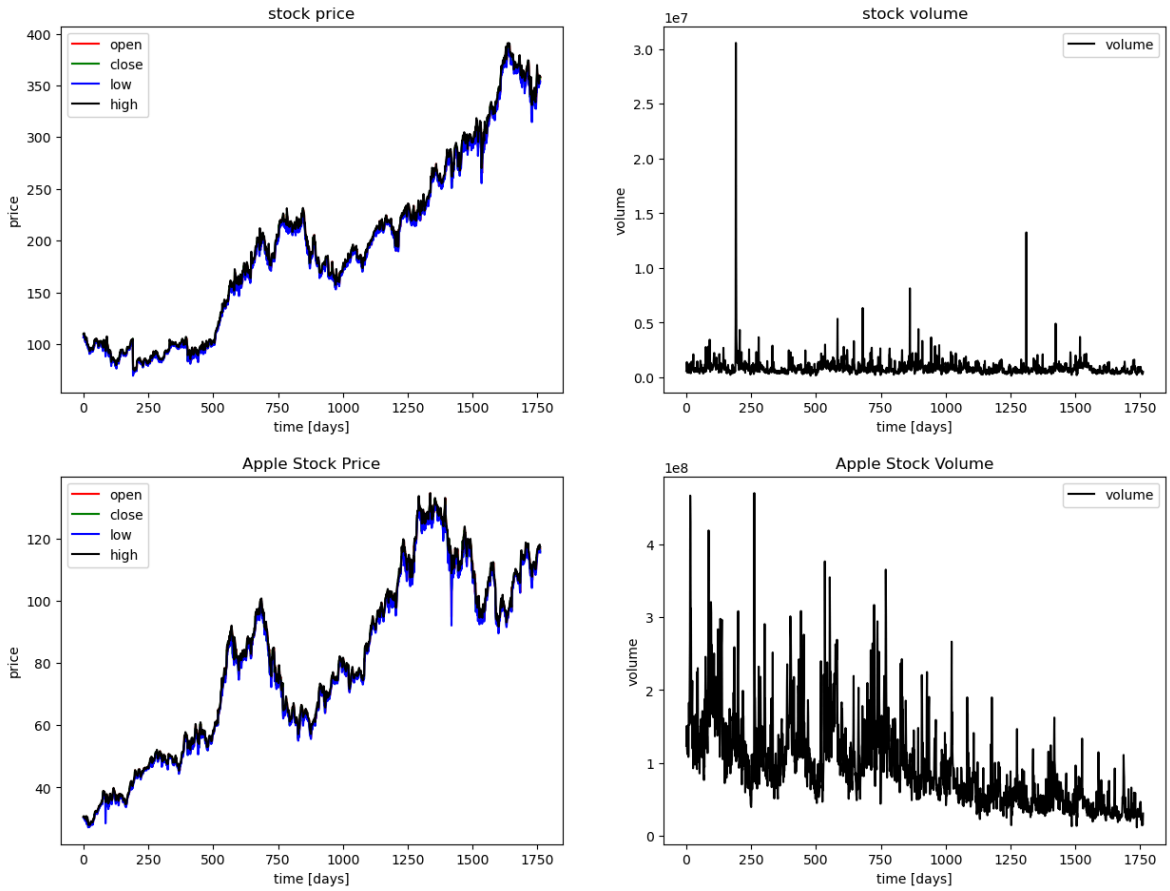
```
# complete the code here
# import all stock prices
# using info, describe, and head function from pandas to get a sense of the data
df = pd.read_csv("prices-split-adjusted.csv", index_col=0)

df.info()
print(df.describe())
print(df.head())

# print the number of different stocks
print('\nnumber of stocks: ', len(list(set(df.symbol))))

# print the symbol of first 10 different stocks

unique_stocks = list(set(df.symbol))

print('\nThe symbols of the first ten stocks: ', unique_stocks[:10])
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 851264 entries, 2016-01-05 to 2016-12-30
Data columns (total 6 columns):
 #   Column  Non-Null Count   Dtype
---  ------  --------------   -----
 0   symbol  851264 non-null  object
 1   open    851264 non-null  float64
 2   close   851264 non-null  float64
 3   low     851264 non-null  float64
 4   high    851264 non-null  float64
 5   volume  851264 non-null  float64
dtypes: float64(5), object(1)
memory usage: 45.5+ MB
                open          close            low           high  \
count  851264.000000  851264.000000  851264.000000  851264.000000
mean       64.993618      65.011913      64.336541      65.639748
std        75.203893      75.201216      74.459518      75.906861
min         1.660000       1.590000       1.500000       1.810000
25%        31.270000      31.292776      30.940001      31.620001
50%        48.459999      48.480000      47.970001      48.959999
75%        75.120003      75.139999      74.400002      75.849998
max      1584.439941    1578.130005    1549.939941    1600.930054

             volume
count  8.512640e+05
mean   5.415113e+06
std    1.249468e+07
min    0.000000e+00
25%    1.221500e+06
50%    2.476250e+06
75%    5.222500e+06
max    8.596434e+08
           symbol        open       close         low        high     volume
date
2016-01-05   WLTW  123.430000  125.839996  122.309998  126.250000  2163600.0
2016-01-06   WLTW  125.239998  119.980003  119.940002  125.540001  2386400.0
2016-01-07   WLTW  116.379997  114.949997  114.930000  119.739998  2489500.0
2016-01-08   WLTW  115.480003  116.620003  113.500000  117.440002  2006300.0
2016-01-11   WLTW  117.010002  114.970001  114.089996  117.330002  1408600.0

number of stocks:  501

The symbols of the first ten stocks:  ['CINF', 'DVA', 'TXN', 'R', 'MTB', 'KSU',
'SWKS', 'LH', 'AIG', 'KIM']
```

In [18]:

```python
# visulization, plot the stock price of one stock in the dataset
plt.figure(figsize=(15, 5));
plt.subplot(1,2,1);
plt.plot(df[df.symbol == 'EQIX'].open.values, color='red', label='open')
plt.plot(df[df.symbol == 'EQIX'].close.values, color='green', label='close')
plt.plot(df[df.symbol == 'EQIX'].low.values, color='blue', label='low')
plt.plot(df[df.symbol == 'EQIX'].high.values, color='black', label='high')
plt.title('stock price')
plt.xlabel('time [days]')
plt.ylabel('price')
plt.legend(loc='best')
#plt.show()

plt.subplot(1,2,2);
plt.plot(df[df.symbol == 'EQIX'].volume.values, color='black', label='volume')
```

```
plt.title('stock volume')
plt.xlabel('time [days]')
plt.ylabel('volume')
plt.legend(loc='best');


# complete the code here
# plot the stock price of Apple "AAPL" in the dataset
plt.figure(figsize=(15, 5));
plt.subplot(1,2,1);
plt.plot(df[df.symbol == 'AAPL'].open.values, color='red', label='open')
plt.plot(df[df.symbol == 'AAPL'].close.values, color='green', label='close')
plt.plot(df[df.symbol == 'AAPL'].low.values, color='blue', label='low')
plt.plot(df[df.symbol == 'AAPL'].high.values, color='black', label='high')
plt.title('Apple Stock Price')
plt.xlabel('time [days]')
plt.ylabel('price')
plt.legend(loc='best')

plt.subplot(1,2,2);
plt.plot(df[df.symbol == 'AAPL'].volume.values, color='black', label='volume')
plt.title('Apple Stock Volume')
plt.xlabel('time [days]')
plt.ylabel('volume')
plt.legend(loc='best');
```



3. Manupulate data

- choose a specific stock
- drop feature: volume
- normalize stock data

- create train, validation and test data sets

In [20]:
```
# split data in 80%/10%/10% train/validation/ test sets
validation_set = 10
test_set = 10

# function for min-max normalization of stock
def normalize_data(df):
    min_max_scaler = sklearn.preprocessing.MinMaxScaler()
    df['open'] = min_max_scaler.fit_transform(df.open.values.reshape(-1,1))
    df['high'] = min_max_scaler.fit_transform(df.high.values.reshape(-1,1))
    df['low'] = min_max_scaler.fit_transform(df.low.values.reshape(-1,1))
    df['close'] = min_max_scaler.fit_transform(df['close'].values.reshape(-1,1))
    return df

# exaplain why do we need to normalize the data
'''
If model accuracy requires that all features have the same scale, we want to mak
When all features are in agreement with each other, the optimization curve becom
'''
```

Out[20]:
```
'\nIf model accuracy requires that all features have the same scale, we want to
make the magnitudes in the distance per feature be the same.\nWhen all features
are in agreement with each other, the optimization curve becomes more round and
smooth, making it easier to optimize on.\n'
```

In [21]:
```
# function to create train, validation, test data given stock data and sequence
def load_data(stock, seq_len, validation_set=10, test_set=10):
    # Convert to numpy array
    data_raw = stock.to_numpy()
    data = []

    # Create all possible sequences of length seq_len
    for index in range(len(data_raw) - seq_len):
        data.append(data_raw[index: index + seq_len])

    data = np.array(data)
    valid_set_size = int(np.round(validation_set / 100 * data.shape[0]))
    test_set_size = int(np.round(test_set / 100 * data.shape[0]))
    train_set_size = data.shape[0] - (valid_set_size + test_set_size)

    x_train = data[:train_set_size, :-1, :]
    y_train = data[:train_set_size, -1, :]

    x_valid = data[train_set_size:train_set_size+valid_set_size, :-1, :]
    y_valid = data[train_set_size:train_set_size+valid_set_size, -1, :]

    x_test = data[train_set_size+valid_set_size:, :-1, :]
    y_test = data[train_set_size+valid_set_size:, -1, :]

    return [x_train, y_train, x_valid, y_valid, x_test, y_test]

# choose one stock
df_stock = df[df.symbol == 'EQIX'].copy()
print(df_stock.head())
df_stock.drop(['symbol'],axis=1,inplace=True)
df_stock.drop(['volume'],axis=1,inplace=True)

cols = list(df_stock.columns.values)
```

```
print('df_stock.columns.values = ', cols)

# normalize stock
df_stock_norm = df_stock.copy()
df_stock_norm = normalize_data(df_stock_norm)

# Usage example
seq_len = 20  # Sequence length
x_train, y_train, x_valid, y_valid, x_test, y_test = load_data(df_stock_norm, se
print('x_train.shape = ', x_train.shape)
print('y_train.shape = ', y_train.shape)
print('x_valid.shape = ', x_valid.shape)
print('y_valid.shape = ', y_valid.shape)
print('x_test.shape = ', x_test.shape)
print('y_test.shape = ', y_test.shape)

# exaplain the meaning of the shape of x_train here
'''
The shape of x_train is (n_samples, n_steps, n_features):
n_samples: The number of training sequences
n_steps: The length of each sequence
n_features: The number of features for each time step
'''
```

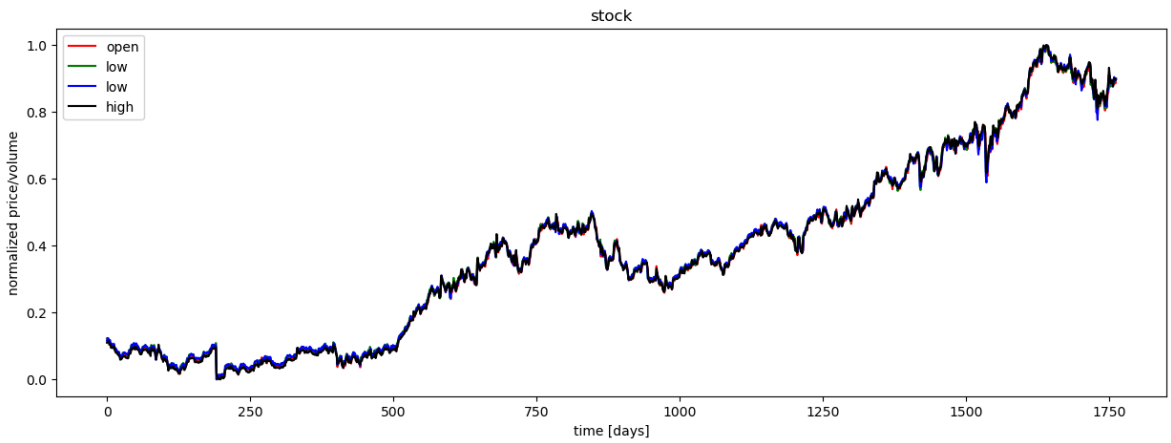
```
           symbol        open       close         low        high     volume
date
2010-01-04   EQIX  106.519997  109.559998  106.510002  109.620003   576300.0
2010-01-05   EQIX  109.589996  108.540001  108.379997  109.589996   681900.0
2010-01-06   EQIX  108.949997  109.529999  108.220001  110.570000  1397500.0
2010-01-07   EQIX  109.250000  107.290001  106.639999  110.349998   797200.0
2010-01-08   EQIX  106.800003  106.769997  105.900002  107.279999   432400.0
df_stock.columns.values =  ['open', 'close', 'low', 'high']
x_train.shape =  (1394, 19, 4)
y_train.shape =  (1394, 4)
x_valid.shape =  (174, 19, 4)
y_valid.shape =  (174, 4)
x_test.shape =  (174, 19, 4)
y_test.shape =  (174, 4)
```

Out[21]:

```
'\nThe shape of x_train is (n_samples, n_steps, n_features):\nn_samples: The nu
mber of training sequences\nn_steps: The length of each sequence \nn_features:
The number of features for each time step\n'
```

In [22]:

```
plt.figure(figsize=(15, 5));
plt.plot(df_stock_norm.open.values, color='red', label='open')
plt.plot(df_stock_norm.close.values, color='green', label='low')
plt.plot(df_stock_norm.low.values, color='blue', label='low')
plt.plot(df_stock_norm.high.values, color='black', label='high')
#plt.plot(df_stock_norm.volume.values, color='gray', label='volume')
plt.title('stock')
plt.xlabel('time [days]')
plt.ylabel('normalized price/volume')
plt.legend(loc='best')
plt.show()
```



4. Model and validate data

- Basic RNN

In [24]:
```
# define the training hyperparameters
n_steps = seq_len-1
n_inputs = 4
n_neurons = 200
n_outputs = 4
n_layers = 2
learning_rate = 0.001
batch_size = 50
n_epochs = 100


# Define the model
model = tf.keras.Sequential([
    tf.keras.layers.InputLayer(input_shape=(n_steps, n_inputs)),  # Input layer
    tf.keras.layers.SimpleRNN(n_neurons, return_sequences=False),  # Simple RNN
    tf.keras.layers.Dense(n_outputs)  # Output layer
])

optimizer=tf.keras.optimizers.Adam(learning_rate=learning_rate)
loss='mse'
# complete the code here
# Compile the model
model.compile(optimizer=optimizer, loss=loss, metrics=['mae'])

# Train the model
history = model.fit(
    x_train, y_train,
    batch_size=batch_size,
    epochs=n_epochs,
    validation_data=(x_valid, y_valid),
    verbose=1
)

# Evaluate the model loss and print it
test_loss, test_mae = model.evaluate(x_test, y_test)
print(f"Test loss: {test_loss:.4f}")
print(f"Test MAE: {test_mae:.4f}")
```

```
E:\Anaconda\Lib\site-packages\keras\src\layers\core\input_layer.py:27: UserWarnin
g: Argument `input_shape` is deprecated. Use `shape` instead.
  warnings.warn(
```

```
Epoch 1/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 4s 37ms/step - loss: 0.0492 - mae: 0.1310 - val_loss:
0.0042 - val_mae: 0.0565
Epoch 2/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 9.9919e-04 - mae: 0.0225 - val_lo
ss: 0.0057 - val_mae: 0.0653
Epoch 3/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 4.2606e-04 - mae: 0.0154 - val_lo
ss: 0.0023 - val_mae: 0.0398
Epoch 4/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 19ms/step - loss: 3.4452e-04 - mae: 0.0140 - val_lo
ss: 0.0017 - val_mae: 0.0346
Epoch 5/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - loss: 3.0730e-04 - mae: 0.0128 - val_lo
ss: 0.0014 - val_mae: 0.0314
Epoch 6/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - loss: 2.9974e-04 - mae: 0.0130 - val_lo
ss: 9.3961e-04 - val_mae: 0.0235
Epoch 7/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 2.3863e-04 - mae: 0.0114 - val_lo
ss: 0.0012 - val_mae: 0.0267
Epoch 8/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - loss: 2.4320e-04 - mae: 0.0113 - val_lo
ss: 8.0722e-04 - val_mae: 0.0210
Epoch 9/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 19ms/step - loss: 2.4516e-04 - mae: 0.0111 - val_lo
ss: 9.4342e-04 - val_mae: 0.0239
Epoch 10/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 2.1480e-04 - mae: 0.0105 - val_lo
ss: 7.8750e-04 - val_mae: 0.0212
Epoch 11/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 19ms/step - loss: 2.2082e-04 - mae: 0.0107 - val_lo
ss: 8.0151e-04 - val_mae: 0.0218
Epoch 12/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 19ms/step - loss: 1.8633e-04 - mae: 0.0100 - val_lo
ss: 7.8135e-04 - val_mae: 0.0213
Epoch 13/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 1.9452e-04 - mae: 0.0101 - val_lo
ss: 6.6198e-04 - val_mae: 0.0183
Epoch 14/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 19ms/step - loss: 1.8181e-04 - mae: 0.0098 - val_lo
ss: 6.6162e-04 - val_mae: 0.0186
Epoch 15/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 1.8284e-04 - mae: 0.0098 - val_lo
ss: 6.4950e-04 - val_mae: 0.0188
Epoch 16/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 16ms/step - loss: 1.8133e-04 - mae: 0.0098 - val_lo
ss: 7.3056e-04 - val_mae: 0.0211
Epoch 17/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 1.7094e-04 - mae: 0.0096 - val_lo
ss: 9.7890e-04 - val_mae: 0.0252
Epoch 18/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 1.8669e-04 - mae: 0.0097 - val_lo
ss: 5.9441e-04 - val_mae: 0.0173
Epoch 19/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 1.6758e-04 - mae: 0.0093 - val_lo
ss: 5.8115e-04 - val_mae: 0.0173
Epoch 20/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 23ms/step - loss: 1.7238e-04 - mae: 0.0094 - val_lo
ss: 8.6147e-04 - val_mae: 0.0239
```

```
Epoch 21/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 21ms/step - loss: 1.6114e-04 - mae: 0.0092 - val_lo
ss: 7.8894e-04 - val_mae: 0.0223
Epoch 22/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 19ms/step - loss: 1.7026e-04 - mae: 0.0096 - val_lo
ss: 6.3635e-04 - val_mae: 0.0182
Epoch 23/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - loss: 1.8245e-04 - mae: 0.0097 - val_lo
ss: 5.9359e-04 - val_mae: 0.0179
Epoch 24/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 19ms/step - loss: 2.0687e-04 - mae: 0.0102 - val_lo
ss: 5.7209e-04 - val_mae: 0.0180
Epoch 25/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 1.5111e-04 - mae: 0.0088 - val_lo
ss: 4.9817e-04 - val_mae: 0.0161
Epoch 26/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 1.4781e-04 - mae: 0.0086 - val_lo
ss: 5.6498e-04 - val_mae: 0.0178
Epoch 27/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 20ms/step - loss: 1.4588e-04 - mae: 0.0086 - val_lo
ss: 5.2154e-04 - val_mae: 0.0169
Epoch 28/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - loss: 1.4234e-04 - mae: 0.0086 - val_lo
ss: 4.7110e-04 - val_mae: 0.0154
Epoch 29/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 20ms/step - loss: 1.3592e-04 - mae: 0.0081 - val_lo
ss: 5.3112e-04 - val_mae: 0.0171
Epoch 30/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 1.4171e-04 - mae: 0.0086 - val_lo
ss: 4.3160e-04 - val_mae: 0.0143
Epoch 31/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 20ms/step - loss: 1.3701e-04 - mae: 0.0085 - val_lo
ss: 5.0493e-04 - val_mae: 0.0171
Epoch 32/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 1.3637e-04 - mae: 0.0082 - val_lo
ss: 6.8779e-04 - val_mae: 0.0208
Epoch 33/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 19ms/step - loss: 1.3073e-04 - mae: 0.0084 - val_lo
ss: 5.5687e-04 - val_mae: 0.0185
Epoch 34/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - loss: 1.4398e-04 - mae: 0.0086 - val_lo
ss: 4.6727e-04 - val_mae: 0.0161
Epoch 35/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 1.4042e-04 - mae: 0.0082 - val_lo
ss: 4.4350e-04 - val_mae: 0.0155
Epoch 36/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 20ms/step - loss: 1.3596e-04 - mae: 0.0084 - val_lo
ss: 6.1052e-04 - val_mae: 0.0197
Epoch 37/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 19ms/step - loss: 1.4959e-04 - mae: 0.0088 - val_lo
ss: 4.1376e-04 - val_mae: 0.0148
Epoch 38/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 20ms/step - loss: 1.1313e-04 - mae: 0.0074 - val_lo
ss: 5.7138e-04 - val_mae: 0.0188
Epoch 39/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 1.2373e-04 - mae: 0.0079 - val_lo
ss: 4.0967e-04 - val_mae: 0.0144
Epoch 40/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 20ms/step - loss: 1.3436e-04 - mae: 0.0081 - val_lo
ss: 4.3823e-04 - val_mae: 0.0157
```

```
Epoch 41/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 1.1197e-04 - mae: 0.0076 - val_lo
ss: 3.8963e-04 - val_mae: 0.0138
Epoch 42/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - loss: 1.3259e-04 - mae: 0.0081 - val_lo
ss: 6.3430e-04 - val_mae: 0.0199
Epoch 43/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 16ms/step - loss: 1.3109e-04 - mae: 0.0082 - val_lo
ss: 4.0702e-04 - val_mae: 0.0149
Epoch 44/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 19ms/step - loss: 1.1067e-04 - mae: 0.0074 - val_lo
ss: 3.9060e-04 - val_mae: 0.0141
Epoch 45/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - loss: 1.1565e-04 - mae: 0.0075 - val_lo
ss: 4.0239e-04 - val_mae: 0.0139
Epoch 46/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 1.6722e-04 - mae: 0.0092 - val_lo
ss: 3.6090e-04 - val_mae: 0.0135
Epoch 47/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 1.1256e-04 - mae: 0.0076 - val_lo
ss: 4.4149e-04 - val_mae: 0.0160
Epoch 48/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 1.0732e-04 - mae: 0.0072 - val_lo
ss: 4.3620e-04 - val_mae: 0.0148
Epoch 49/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - loss: 1.3639e-04 - mae: 0.0083 - val_lo
ss: 3.7780e-04 - val_mae: 0.0141
Epoch 50/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 19ms/step - loss: 1.1328e-04 - mae: 0.0071 - val_lo
ss: 3.9088e-04 - val_mae: 0.0141
Epoch 51/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 1.0713e-04 - mae: 0.0073 - val_lo
ss: 3.9842e-04 - val_mae: 0.0134
Epoch 52/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 1.2108e-04 - mae: 0.0077 - val_lo
ss: 3.4797e-04 - val_mae: 0.0134
Epoch 53/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 9.1948e-05 - mae: 0.0067 - val_lo
ss: 3.7030e-04 - val_mae: 0.0136
Epoch 54/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 19ms/step - loss: 1.0189e-04 - mae: 0.0072 - val_lo
ss: 3.4155e-04 - val_mae: 0.0133
Epoch 55/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 1.0208e-04 - mae: 0.0072 - val_lo
ss: 3.7364e-04 - val_mae: 0.0145
Epoch 56/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 20ms/step - loss: 8.5938e-05 - mae: 0.0067 - val_lo
ss: 3.3644e-04 - val_mae: 0.0132
Epoch 57/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 19ms/step - loss: 1.0292e-04 - mae: 0.0071 - val_lo
ss: 3.3408e-04 - val_mae: 0.0130
Epoch 58/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 19ms/step - loss: 1.0314e-04 - mae: 0.0072 - val_lo
ss: 3.7100e-04 - val_mae: 0.0143
Epoch 59/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - loss: 1.2007e-04 - mae: 0.0079 - val_lo
ss: 3.3091e-04 - val_mae: 0.0127
Epoch 60/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 1.0365e-04 - mae: 0.0073 - val_lo
ss: 3.2837e-04 - val_mae: 0.0122
```

```
Epoch 61/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 9.7892e-05 - mae: 0.0069 - val_lo
ss: 3.1683e-04 - val_mae: 0.0129
Epoch 62/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - loss: 9.3892e-05 - mae: 0.0067 - val_lo
ss: 3.2596e-04 - val_mae: 0.0123
Epoch 63/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 19ms/step - loss: 1.0259e-04 - mae: 0.0071 - val_lo
ss: 3.7904e-04 - val_mae: 0.0150
Epoch 64/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 8.8856e-05 - mae: 0.0066 - val_lo
ss: 4.2409e-04 - val_mae: 0.0165
Epoch 65/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 1.0747e-04 - mae: 0.0071 - val_lo
ss: 3.4959e-04 - val_mae: 0.0142
Epoch 66/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 19ms/step - loss: 9.0287e-05 - mae: 0.0065 - val_lo
ss: 2.9945e-04 - val_mae: 0.0122
Epoch 67/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 9.1981e-05 - mae: 0.0066 - val_lo
ss: 3.1231e-04 - val_mae: 0.0132
Epoch 68/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - loss: 8.5763e-05 - mae: 0.0065 - val_lo
ss: 3.1741e-04 - val_mae: 0.0124
Epoch 69/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 26ms/step - loss: 9.4189e-05 - mae: 0.0067 - val_lo
ss: 4.1461e-04 - val_mae: 0.0159
Epoch 70/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 37ms/step - loss: 9.3098e-05 - mae: 0.0068 - val_lo
ss: 4.2366e-04 - val_mae: 0.0167
Epoch 71/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - loss: 1.0541e-04 - mae: 0.0072 - val_lo
ss: 2.8923e-04 - val_mae: 0.0120
Epoch 72/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 8.9696e-05 - mae: 0.0065 - val_lo
ss: 2.9191e-04 - val_mae: 0.0120
Epoch 73/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - loss: 8.6219e-05 - mae: 0.0066 - val_lo
ss: 2.8158e-04 - val_mae: 0.0115
Epoch 74/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - loss: 7.6067e-05 - mae: 0.0062 - val_lo
ss: 2.8800e-04 - val_mae: 0.0118
Epoch 75/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 23ms/step - loss: 8.9300e-05 - mae: 0.0064 - val_lo
ss: 2.7194e-04 - val_mae: 0.0116
Epoch 76/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - loss: 8.3230e-05 - mae: 0.0063 - val_lo
ss: 3.0179e-04 - val_mae: 0.0121
Epoch 77/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 19ms/step - loss: 1.0512e-04 - mae: 0.0073 - val_lo
ss: 3.5457e-04 - val_mae: 0.0137
Epoch 78/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 8.4595e-05 - mae: 0.0065 - val_lo
ss: 3.6895e-04 - val_mae: 0.0146
Epoch 79/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - loss: 9.3351e-05 - mae: 0.0067 - val_lo
ss: 5.0564e-04 - val_mae: 0.0185
Epoch 80/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 9.7215e-05 - mae: 0.0071 - val_lo
ss: 3.1555e-04 - val_mae: 0.0131
```

```
Epoch 81/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 8.0151e-05 - mae: 0.0063 - val_lo
ss: 2.6739e-04 - val_mae: 0.0115
Epoch 82/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 9.0782e-05 - mae: 0.0066 - val_lo
ss: 2.9384e-04 - val_mae: 0.0121
Epoch 83/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 19ms/step - loss: 8.7003e-05 - mae: 0.0063 - val_lo
ss: 3.7170e-04 - val_mae: 0.0153
Epoch 84/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 7.8031e-05 - mae: 0.0060 - val_lo
ss: 3.6238e-04 - val_mae: 0.0149
Epoch 85/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 8.2054e-05 - mae: 0.0062 - val_lo
ss: 3.1618e-04 - val_mae: 0.0138
Epoch 86/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 16ms/step - loss: 7.2999e-05 - mae: 0.0061 - val_lo
ss: 3.0854e-04 - val_mae: 0.0125
Epoch 87/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 21ms/step - loss: 8.5536e-05 - mae: 0.0063 - val_lo
ss: 2.7657e-04 - val_mae: 0.0119
Epoch 88/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 22ms/step - loss: 7.3988e-05 - mae: 0.0060 - val_lo
ss: 2.6769e-04 - val_mae: 0.0120
Epoch 89/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 20ms/step - loss: 8.0022e-05 - mae: 0.0061 - val_lo
ss: 3.3092e-04 - val_mae: 0.0142
Epoch 90/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 23ms/step - loss: 8.1002e-05 - mae: 0.0061 - val_lo
ss: 2.6835e-04 - val_mae: 0.0116
Epoch 91/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 25ms/step - loss: 7.7175e-05 - mae: 0.0061 - val_lo
ss: 2.9975e-04 - val_mae: 0.0131
Epoch 92/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - loss: 8.0589e-05 - mae: 0.0063 - val_lo
ss: 2.6477e-04 - val_mae: 0.0119
Epoch 93/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 22ms/step - loss: 7.9481e-05 - mae: 0.0061 - val_lo
ss: 4.1253e-04 - val_mae: 0.0157
Epoch 94/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 21ms/step - loss: 8.7554e-05 - mae: 0.0065 - val_lo
ss: 3.2940e-04 - val_mae: 0.0139
Epoch 95/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 30ms/step - loss: 7.2334e-05 - mae: 0.0059 - val_lo
ss: 3.3441e-04 - val_mae: 0.0131
Epoch 96/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 19ms/step - loss: 7.4552e-05 - mae: 0.0060 - val_lo
ss: 2.4776e-04 - val_mae: 0.0109
Epoch 97/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 19ms/step - loss: 7.1044e-05 - mae: 0.0060 - val_lo
ss: 2.9901e-04 - val_mae: 0.0123
Epoch 98/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 9.2765e-05 - mae: 0.0067 - val_lo
ss: 2.5994e-04 - val_mae: 0.0115
Epoch 99/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 16ms/step - loss: 7.3787e-05 - mae: 0.0060 - val_lo
ss: 3.3281e-04 - val_mae: 0.0141
Epoch 100/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 8.6254e-05 - mae: 0.0068 - val_lo
ss: 2.4778e-04 - val_mae: 0.0108
```

```
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 1.3554e-04 - mae: 0.0088
Test loss: 0.0002
Test MAE: 0.0095
```

- RNN+LSTM

In [26]:

```python
# explore LSTM
# RNN+LSTM in tensorflow

# Define the model
model1 = tf.keras.Sequential([
    tf.keras.layers.InputLayer(input_shape=(n_steps, n_inputs)),
    tf.keras.layers.LSTM(n_neurons, return_sequences=True),
    tf.keras.layers.LSTM(n_neurons),
    tf.keras.layers.Dense(n_outputs)
])

optimizer=tf.keras.optimizers.Adam(learning_rate=learning_rate)
loss='mse'
# complete the code here
# Compile the model
model1.compile(optimizer=optimizer, loss=loss, metrics=['mae'])

# Train the model
history1 = model1.fit(
    x_train, y_train,
    batch_size=batch_size,
    epochs=n_epochs,
    validation_data=(x_valid, y_valid),
    verbose=1
)

# Evaluate the model
test_loss1, test_mae1 = model1.evaluate(x_test, y_test)
print(f"LSTM Test loss: {test_loss1:.4f}")
print(f"LSTM Test MAE: {test_mae1:.4f}")
```

```
E:\Anaconda\Lib\site-packages\keras\src\layers\core\input_layer.py:27: UserWarnin
g: Argument `input_shape` is deprecated. Use `shape` instead.
  warnings.warn(
```

```
Epoch 1/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 9s 116ms/step - loss: 0.0298 - mae: 0.1198 - val_loss:
0.0105 - val_mae: 0.0885
Epoch 2/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 87ms/step - loss: 7.7399e-04 - mae: 0.0210 - val_lo
ss: 0.0012 - val_mae: 0.0278
Epoch 3/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 3s 95ms/step - loss: 3.3137e-04 - mae: 0.0139 - val_lo
ss: 0.0013 - val_mae: 0.0296
Epoch 4/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 3s 96ms/step - loss: 3.4125e-04 - mae: 0.0137 - val_lo
ss: 0.0013 - val_mae: 0.0300
Epoch 5/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 5s 85ms/step - loss: 3.0849e-04 - mae: 0.0133 - val_lo
ss: 0.0012 - val_mae: 0.0285
Epoch 6/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 82ms/step - loss: 2.9831e-04 - mae: 0.0130 - val_lo
ss: 9.4661e-04 - val_mae: 0.0230
Epoch 7/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 84ms/step - loss: 2.9166e-04 - mae: 0.0130 - val_lo
ss: 0.0010 - val_mae: 0.0245
Epoch 8/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 79ms/step - loss: 3.2392e-04 - mae: 0.0134 - val_lo
ss: 0.0011 - val_mae: 0.0265
Epoch 9/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 77ms/step - loss: 3.0638e-04 - mae: 0.0131 - val_lo
ss: 9.8923e-04 - val_mae: 0.0244
Epoch 10/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 3s 88ms/step - loss: 2.7309e-04 - mae: 0.0127 - val_lo
ss: 9.1283e-04 - val_mae: 0.0217
Epoch 11/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 84ms/step - loss: 2.8688e-04 - mae: 0.0125 - val_lo
ss: 8.9570e-04 - val_mae: 0.0221
Epoch 12/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 85ms/step - loss: 2.7700e-04 - mae: 0.0124 - val_lo
ss: 0.0011 - val_mae: 0.0257
Epoch 13/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 86ms/step - loss: 2.7194e-04 - mae: 0.0124 - val_lo
ss: 8.8116e-04 - val_mae: 0.0214
Epoch 14/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 86ms/step - loss: 2.4434e-04 - mae: 0.0115 - val_lo
ss: 8.6913e-04 - val_mae: 0.0218
Epoch 15/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 79ms/step - loss: 2.4432e-04 - mae: 0.0118 - val_lo
ss: 8.8068e-04 - val_mae: 0.0210
Epoch 16/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 77ms/step - loss: 2.4636e-04 - mae: 0.0116 - val_lo
ss: 0.0013 - val_mae: 0.0295
Epoch 17/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 79ms/step - loss: 2.7350e-04 - mae: 0.0122 - val_lo
ss: 0.0011 - val_mae: 0.0272
Epoch 18/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 85ms/step - loss: 2.5037e-04 - mae: 0.0118 - val_lo
ss: 8.6437e-04 - val_mae: 0.0209
Epoch 19/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 84ms/step - loss: 3.2109e-04 - mae: 0.0131 - val_lo
ss: 9.9059e-04 - val_mae: 0.0247
Epoch 20/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 85ms/step - loss: 2.5164e-04 - mae: 0.0116 - val_lo
ss: 8.2882e-04 - val_mae: 0.0215
```

```
Epoch 21/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 85ms/step - loss: 1.9789e-04 - mae: 0.0106 - val_lo
ss: 8.4197e-04 - val_mae: 0.0204
Epoch 22/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 80ms/step - loss: 2.1354e-04 - mae: 0.0110 - val_lo
ss: 0.0010 - val_mae: 0.0259
Epoch 23/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 81ms/step - loss: 2.5313e-04 - mae: 0.0119 - val_lo
ss: 7.7974e-04 - val_mae: 0.0206
Epoch 24/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 70ms/step - loss: 2.2552e-04 - mae: 0.0108 - val_lo
ss: 8.2753e-04 - val_mae: 0.0204
Epoch 25/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 78ms/step - loss: 1.9984e-04 - mae: 0.0104 - val_lo
ss: 7.5116e-04 - val_mae: 0.0199
Epoch 26/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 87ms/step - loss: 2.0659e-04 - mae: 0.0105 - val_lo
ss: 7.9668e-04 - val_mae: 0.0195
Epoch 27/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 81ms/step - loss: 1.9807e-04 - mae: 0.0105 - val_lo
ss: 8.0793e-04 - val_mae: 0.0215
Epoch 28/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 84ms/step - loss: 2.0595e-04 - mae: 0.0106 - val_lo
ss: 0.0012 - val_mae: 0.0288
Epoch 29/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 74ms/step - loss: 2.2700e-04 - mae: 0.0112 - val_lo
ss: 7.4629e-04 - val_mae: 0.0191
Epoch 30/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 79ms/step - loss: 2.4634e-04 - mae: 0.0115 - val_lo
ss: 7.9562e-04 - val_mae: 0.0195
Epoch 31/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 80ms/step - loss: 2.3421e-04 - mae: 0.0110 - val_lo
ss: 6.8260e-04 - val_mae: 0.0189
Epoch 32/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 3s 89ms/step - loss: 1.7917e-04 - mae: 0.0100 - val_lo
ss: 7.3040e-04 - val_mae: 0.0203
Epoch 33/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 81ms/step - loss: 1.7982e-04 - mae: 0.0100 - val_lo
ss: 8.0810e-04 - val_mae: 0.0198
Epoch 34/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 78ms/step - loss: 2.0615e-04 - mae: 0.0105 - val_lo
ss: 7.1626e-04 - val_mae: 0.0200
Epoch 35/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 79ms/step - loss: 2.0008e-04 - mae: 0.0106 - val_lo
ss: 7.4786e-04 - val_mae: 0.0207
Epoch 36/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 73ms/step - loss: 1.8000e-04 - mae: 0.0099 - val_lo
ss: 8.2899e-04 - val_mae: 0.0230
Epoch 37/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 80ms/step - loss: 1.8934e-04 - mae: 0.0100 - val_lo
ss: 6.3153e-04 - val_mae: 0.0173
Epoch 38/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 83ms/step - loss: 1.9350e-04 - mae: 0.0101 - val_lo
ss: 7.2739e-04 - val_mae: 0.0211
Epoch 39/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 86ms/step - loss: 2.0265e-04 - mae: 0.0102 - val_lo
ss: 7.1312e-04 - val_mae: 0.0206
Epoch 40/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 3s 82ms/step - loss: 1.5923e-04 - mae: 0.0093 - val_lo
ss: 6.5005e-04 - val_mae: 0.0192
```

```
Epoch 41/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 3s 84ms/step - loss: 1.7239e-04 - mae: 0.0097 - val_lo
ss: 6.9450e-04 - val_mae: 0.0192
Epoch 42/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 79ms/step - loss: 1.7958e-04 - mae: 0.0100 - val_lo
ss: 5.6763e-04 - val_mae: 0.0173
Epoch 43/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 74ms/step - loss: 1.6066e-04 - mae: 0.0092 - val_lo
ss: 6.7632e-04 - val_mae: 0.0201
Epoch 44/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 83ms/step - loss: 1.5643e-04 - mae: 0.0092 - val_lo
ss: 5.6232e-04 - val_mae: 0.0171
Epoch 45/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 84ms/step - loss: 1.7149e-04 - mae: 0.0098 - val_lo
ss: 5.7602e-04 - val_mae: 0.0179
Epoch 46/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 75ms/step - loss: 1.7805e-04 - mae: 0.0097 - val_lo
ss: 5.9421e-04 - val_mae: 0.0165
Epoch 47/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 78ms/step - loss: 1.6872e-04 - mae: 0.0095 - val_lo
ss: 5.2521e-04 - val_mae: 0.0162
Epoch 48/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 82ms/step - loss: 2.0197e-04 - mae: 0.0105 - val_lo
ss: 0.0010 - val_mae: 0.0275
Epoch 49/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 80ms/step - loss: 1.7714e-04 - mae: 0.0100 - val_lo
ss: 5.1566e-04 - val_mae: 0.0155
Epoch 50/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 73ms/step - loss: 1.9887e-04 - mae: 0.0100 - val_lo
ss: 5.0423e-04 - val_mae: 0.0162
Epoch 51/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 83ms/step - loss: 1.5187e-04 - mae: 0.0089 - val_lo
ss: 4.8776e-04 - val_mae: 0.0149
Epoch 52/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 80ms/step - loss: 1.5600e-04 - mae: 0.0089 - val_lo
ss: 4.3959e-04 - val_mae: 0.0145
Epoch 53/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 83ms/step - loss: 1.4787e-04 - mae: 0.0088 - val_lo
ss: 5.4557e-04 - val_mae: 0.0178
Epoch 54/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 80ms/step - loss: 1.4733e-04 - mae: 0.0087 - val_lo
ss: 4.5053e-04 - val_mae: 0.0156
Epoch 55/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 72ms/step - loss: 1.4076e-04 - mae: 0.0085 - val_lo
ss: 4.3653e-04 - val_mae: 0.0154
Epoch 56/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 48ms/step - loss: 1.2666e-04 - mae: 0.0080 - val_lo
ss: 4.0612e-04 - val_mae: 0.0136
Epoch 57/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 47ms/step - loss: 1.4071e-04 - mae: 0.0084 - val_lo
ss: 3.8592e-04 - val_mae: 0.0136
Epoch 58/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 49ms/step - loss: 1.2622e-04 - mae: 0.0081 - val_lo
ss: 5.7398e-04 - val_mae: 0.0196
Epoch 59/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 52ms/step - loss: 1.2466e-04 - mae: 0.0081 - val_lo
ss: 3.9061e-04 - val_mae: 0.0142
Epoch 60/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 48ms/step - loss: 1.3894e-04 - mae: 0.0084 - val_lo
ss: 3.4757e-04 - val_mae: 0.0130
```

```
Epoch 61/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 49ms/step - loss: 1.5419e-04 - mae: 0.0087 - val_lo
ss: 3.8194e-04 - val_mae: 0.0144
Epoch 62/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 49ms/step - loss: 1.2431e-04 - mae: 0.0081 - val_lo
ss: 3.6673e-04 - val_mae: 0.0141
Epoch 63/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 3s 54ms/step - loss: 1.2407e-04 - mae: 0.0080 - val_lo
ss: 4.7774e-04 - val_mae: 0.0155
Epoch 64/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 54ms/step - loss: 1.4012e-04 - mae: 0.0085 - val_lo
ss: 6.4063e-04 - val_mae: 0.0216
Epoch 65/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 49ms/step - loss: 1.2615e-04 - mae: 0.0082 - val_lo
ss: 3.0260e-04 - val_mae: 0.0122
Epoch 66/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 47ms/step - loss: 1.0780e-04 - mae: 0.0074 - val_lo
ss: 3.8304e-04 - val_mae: 0.0138
Epoch 67/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 51ms/step - loss: 1.0795e-04 - mae: 0.0074 - val_lo
ss: 3.3777e-04 - val_mae: 0.0133
Epoch 68/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 48ms/step - loss: 1.3123e-04 - mae: 0.0078 - val_lo
ss: 3.6219e-04 - val_mae: 0.0145
Epoch 69/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 48ms/step - loss: 1.1285e-04 - mae: 0.0073 - val_lo
ss: 3.0128e-04 - val_mae: 0.0127
Epoch 70/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 55ms/step - loss: 1.0106e-04 - mae: 0.0072 - val_lo
ss: 2.9582e-04 - val_mae: 0.0124
Epoch 71/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 48ms/step - loss: 1.1863e-04 - mae: 0.0074 - val_lo
ss: 4.0581e-04 - val_mae: 0.0163
Epoch 72/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 48ms/step - loss: 1.1026e-04 - mae: 0.0074 - val_lo
ss: 4.1610e-04 - val_mae: 0.0163
Epoch 73/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 48ms/step - loss: 1.1394e-04 - mae: 0.0078 - val_lo
ss: 3.4875e-04 - val_mae: 0.0135
Epoch 74/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 51ms/step - loss: 1.0359e-04 - mae: 0.0073 - val_lo
ss: 2.4657e-04 - val_mae: 0.0112
Epoch 75/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 49ms/step - loss: 1.3367e-04 - mae: 0.0079 - val_lo
ss: 2.7217e-04 - val_mae: 0.0121
Epoch 76/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - loss: 1.0872e-04 - mae: 0.0071 - val_lo
ss: 3.8617e-04 - val_mae: 0.0151
Epoch 77/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 47ms/step - loss: 1.1743e-04 - mae: 0.0080 - val_lo
ss: 3.1332e-04 - val_mae: 0.0132
Epoch 78/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 51ms/step - loss: 9.3262e-05 - mae: 0.0068 - val_lo
ss: 4.4149e-04 - val_mae: 0.0174
Epoch 79/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 3s 47ms/step - loss: 9.8926e-05 - mae: 0.0069 - val_lo
ss: 3.6109e-04 - val_mae: 0.0152
Epoch 80/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 50ms/step - loss: 1.2283e-04 - mae: 0.0081 - val_lo
ss: 2.6941e-04 - val_mae: 0.0123
```

```
Epoch 81/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 57ms/step - loss: 8.4504e-05 - mae: 0.0065 - val_lo
ss: 2.3798e-04 - val_mae: 0.0113
Epoch 82/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 57ms/step - loss: 8.0004e-05 - mae: 0.0061 - val_lo
ss: 2.2410e-04 - val_mae: 0.0106
Epoch 83/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 47ms/step - loss: 8.7509e-05 - mae: 0.0066 - val_lo
ss: 2.5158e-04 - val_mae: 0.0121
Epoch 84/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - loss: 8.5333e-05 - mae: 0.0065 - val_lo
ss: 2.9251e-04 - val_mae: 0.0129
Epoch 85/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 47ms/step - loss: 8.4737e-05 - mae: 0.0064 - val_lo
ss: 2.1410e-04 - val_mae: 0.0106
Epoch 86/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 3s 54ms/step - loss: 8.0779e-05 - mae: 0.0062 - val_lo
ss: 2.3642e-04 - val_mae: 0.0110
Epoch 87/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 52ms/step - loss: 7.5210e-05 - mae: 0.0060 - val_lo
ss: 2.3557e-04 - val_mae: 0.0117
Epoch 88/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 51ms/step - loss: 9.1189e-05 - mae: 0.0064 - val_lo
ss: 1.9403e-04 - val_mae: 0.0101
Epoch 89/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - loss: 9.0541e-05 - mae: 0.0062 - val_lo
ss: 2.6843e-04 - val_mae: 0.0121
Epoch 90/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 55ms/step - loss: 9.4906e-05 - mae: 0.0066 - val_lo
ss: 2.8861e-04 - val_mae: 0.0135
Epoch 91/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 48ms/step - loss: 8.8226e-05 - mae: 0.0068 - val_lo
ss: 5.2340e-04 - val_mae: 0.0198
Epoch 92/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 47ms/step - loss: 9.9509e-05 - mae: 0.0070 - val_lo
ss: 3.0693e-04 - val_mae: 0.0133
Epoch 93/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 50ms/step - loss: 1.0117e-04 - mae: 0.0074 - val_lo
ss: 1.9347e-04 - val_mae: 0.0099
Epoch 94/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 47ms/step - loss: 8.1123e-05 - mae: 0.0061 - val_lo
ss: 2.4482e-04 - val_mae: 0.0122
Epoch 95/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 48ms/step - loss: 8.7797e-05 - mae: 0.0062 - val_lo
ss: 2.0283e-04 - val_mae: 0.0102
Epoch 96/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 49ms/step - loss: 7.8566e-05 - mae: 0.0058 - val_lo
ss: 2.0221e-04 - val_mae: 0.0101
Epoch 97/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 50ms/step - loss: 7.3801e-05 - mae: 0.0057 - val_lo
ss: 2.0318e-04 - val_mae: 0.0099
Epoch 98/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 48ms/step - loss: 8.5800e-05 - mae: 0.0065 - val_lo
ss: 1.8886e-04 - val_mae: 0.0100
Epoch 99/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 48ms/step - loss: 6.5181e-05 - mae: 0.0056 - val_lo
ss: 2.0167e-04 - val_mae: 0.0100
Epoch 100/100
28/28 ━━━━━━━━━━━━━━━━━━━━ 3s 51ms/step - loss: 6.2860e-05 - mae: 0.0055 - val_lo
ss: 2.1030e-04 - val_mae: 0.0108
```

```
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - loss: 1.7525e-04 - mae: 0.0107
LSTM Test loss: 0.0002
LSTM Test MAE: 0.0113
```

5. Prediction

Use one of previous models to predict the stocks.

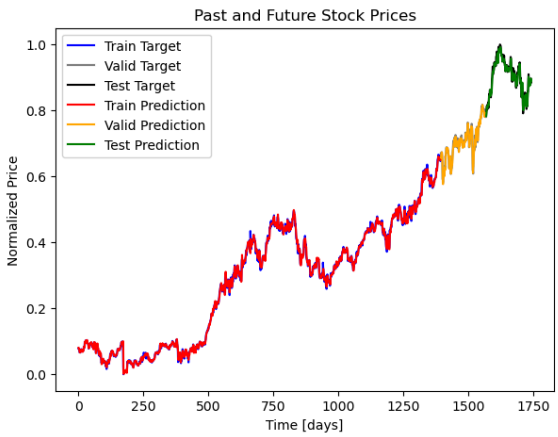
In [28]:

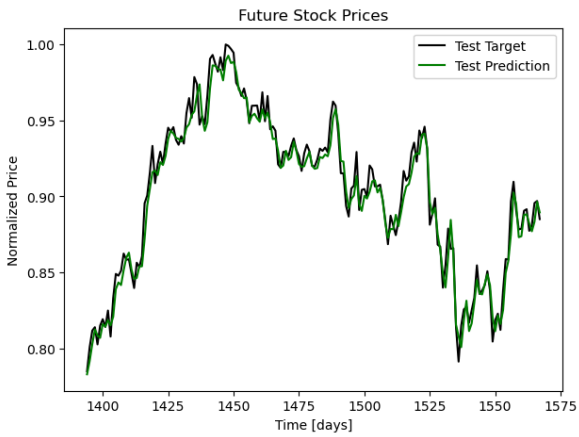
```
ft = 0  # Feature index: 0 = open, 1 = close, 2 = highest, 3 = lowest

# Predictions (replace these with model.predict calls)
y_train_pred = model.predict(x_train)
y_valid_pred = model.predict(x_valid)
y_test_pred = model.predict(x_test)

## Show predictions
plt.figure(figsize=(15, 5))

# Subplot 1: Past and future stock prices
plt.subplot(1, 2, 1)
plt.plot(np.arange(y_train.shape[0]), y_train[:, ft], color='blue', label='Train
plt.plot(np.arange(y_train.shape[0], y_train.shape[0] + y_valid.shape[0]),
         y_valid[:, ft], color='gray', label='Valid Target')
plt.plot(np.arange(y_train.shape[0] + y_valid.shape[0],
                   y_train.shape[0] + y_valid.shape[0] + y_test.shape[0]),
         y_test[:, ft], color='black', label='Test Target')

plt.plot(np.arange(y_train_pred.shape[0]), y_train_pred[:, ft], color='red', lab
plt.plot(np.arange(y_train_pred.shape[0], y_train_pred.shape[0] + y_valid_pred.s
         y_valid_pred[:, ft], color='orange', label='Valid Prediction')
plt.plot(np.arange(y_train_pred.shape[0] + y_valid_pred.shape[0],
                   y_train_pred.shape[0] + y_valid_pred.shape[0] + y_test_pred.s
         y_test_pred[:, ft], color='green', label='Test Prediction')

plt.title('Past and Future Stock Prices')
plt.xlabel('Time [days]')
plt.ylabel('Normalized Price')
plt.legend(loc='best')

# Subplot 2: Future stock prices
plt.subplot(1, 2, 2)
plt.plot(np.arange(y_train.shape[0], y_train.shape[0] + y_test.shape[0]),
         y_test[:, ft], color='black', label='Test Target')
plt.plot(np.arange(y_train_pred.shape[0], y_train_pred.shape[0] + y_test_pred.sh
         y_test_pred[:, ft], color='green', label='Test Prediction')

plt.title('Future Stock Prices')
plt.xlabel('Time [days]')
plt.ylabel('Normalized Price')
plt.legend(loc='best')
```

```
44/44 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step
```

Out[28]:

```
<matplotlib.legend.Legend at 0x2c5612e3890>
```





In [29]:
```
# when you finish, please use 2pdf.com to transform this ipynb to pdf, and submi
# the deadline is 2025-3-21 17:00
```